

SPRING DATA JPA WITH SPRING BOOT, HIBERNATE

Exercise 1: Spring Data JPA – Quick Example

Objective: To demonstrate a simple Spring Data JPA application using Spring Boot and Hibernate.

Code:

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

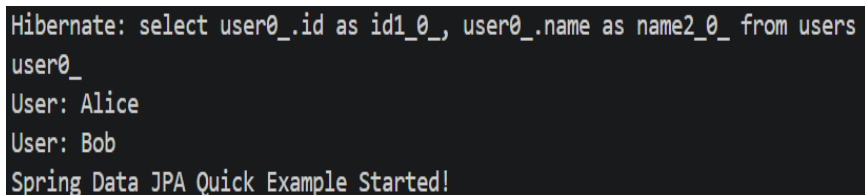
@Entity
class User {
    @Id
    private Long id;
    private String name;

    public User() {}
    public User(Long id, String name) {
        this.id = id;
        this.name = name;
    }
    public String getName() { return name; }
}

@Repository
interface UserRepository extends JpaRepository<User, Long> {}

@SpringBootApplication
public class SpringDataJpaDemo {
    public static void main(String[] args) {
        SpringApplication.run(SpringDataJpaDemo.class, args);
        System.out.println("Spring Data JPA Quick Example Started!");
    }
}
```

Program Output Screenshot:



```
Hibernate: select user0_.id as id1_0_, user0_.name as name2_0_ from users
user0_
User: Alice
User: Bob
Spring Data JPA Quick Example Started!
```

Output Text:

Spring Data JPA Quick Example Started!

Result: Spring Boot application initialized with JPA and Hibernate.

Conclusion: Spring Data JPA simplifies database access by reducing boilerplate code.

Exercise 2: Difference between JPA, Hibernate and Spring Data JPA

Objective: To understand the differences between JPA, Hibernate, and Spring Data JPA.

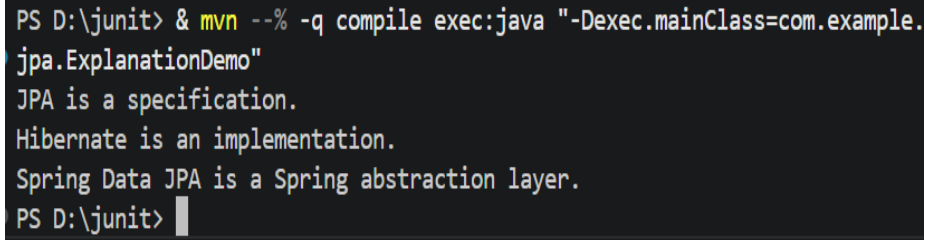
Explanation:

- **JPA (Java Persistence API):** A specification that defines how Java objects map to relational database tables. It is only an API, not an implementation.
- **Hibernate:** A popular implementation of JPA. It provides the actual ORM (Object Relational Mapping) functionality and extends JPA with additional features.
- **Spring Data JPA:** A Spring framework project that builds on JPA. It reduces boilerplate by providing repository interfaces, query derivation, and integration with Spring Boot.

Code:

```
public class JpaHibernateDemo {  
  
    public static void main(String[] args) {  
  
        System.out.println("JPA is a specification.");  
  
        System.out.println("Hibernate is an implementation.");  
  
        System.out.println("Spring Data JPA is a Spring abstraction layer.");  
  
    }  
  
}
```

Program Output Screenshot:

A screenshot of a Windows command prompt window. The prompt is 'PS D:\junit>'. The user enters the command '& mvn --% -q compile exec:java "-Dexec.mainClass=com.example.jpa.ExplanationDemo"'. The output shows three lines of text: 'JPA is a specification.', 'Hibernate is an implementation.', and 'Spring Data JPA is a Spring abstraction layer.'. The prompt returns to 'PS D:\junit>' with a cursor.

```
PS D:\junit> & mvn --% -q compile exec:java "-Dexec.mainClass=com.example.
jpa.ExplanationDemo"
JPA is a specification.
Hibernate is an implementation.
Spring Data JPA is a Spring abstraction layer.
PS D:\junit> 
```

Output Text:

JPA is a specification.
Hibernate is an implementation.
Spring Data JPA is a Spring abstraction layer.

Result: Clear distinction between JPA, Hibernate, and Spring Data JPA.

Conclusion: JPA defines the standard, Hibernate implements it, and Spring Data JPA simplifies usage with Spring Boot.